《Python 数据分析编程》课程项目报告（拟稿）

基于时间序列验证与特征消融的城市短期电力负荷预测研究

目录

1 引言 4

2 数据介绍 5

3 数据清洗与预处理 7

4 负荷的时序规律 9

5 天气因素与负荷的关系 13

6 短期负荷预测：模型、消融与误差分析 15

7 总结与结论 23

8 小组成员分工与个人承担 25

9 附录 25

摘　　要

电力负荷的短期预测是电网调度的基础输入。本项目以某城市 2017 年 1 月 1 日至 12 月 30 日的电力负荷监测数据为对象（共 364 天、52,416 条记录、10 分钟采样、已记录时段内无缺失），包含三个计量区域的负荷与温度、湿度、风速等气象观测。工作依次做了数据清洗、规律探索、预测建模和误差分析。

三个区域有**共同的日周期形态**：凌晨低谷、午间平台、晚间高峰，峰谷比 1.89–2.23；温度是唯一在三区都稳定正相关的气象因子。预测任务定为提前 1 小时（6 个采样步），数据按时间顺序以 70%/15%/15% 划分，并在训练、验证集末尾各剔除 6 个起点；比较**历史同期基线、线性回归和梯度提升树**，另用特征消融检验天气信息能带来多少增益。

测试集上，线性回归精度最高，三区合并 MAE/RMSE 为 514.09/767.16，较历史同期基线（1050.68/1643.67）下降 51.1%/53.3%，较梯度提升树（930.62/1258.73）低 44.8%/39.0%；梯度提升树在三个区域也都优于基线。起作用的**关键特征是与预测目标时刻对齐的周同期负荷 y(t−1002)**：只加这一项，线性回归三区测试集 RMSE 就下降 52.5%–69.0%。天气特征的增益因模型和区域而异：梯度提升树三区全部受益（RMSE 下降 0.4%–17.9%），线性回归只在区域 1 下降 11.0%，区域 2、3 基本持平。按训练集目标负荷的九十分位划分高峰样本后，区域 2 高峰时段误差约为普通时段的 2.1 倍，且模型系统性低估峰值；区域 1、3 的高峰测试样本太少，无法下结论。据此建议短期调度优先采用含周同期特征的线性模型，并在高峰时段预留上调裕度。

关键词：电力负荷预测；时间序列；特征消融；历史同期基线；梯度提升树

1　引言

电力负荷预测是电网运行调度的基本环节：机组组合、备用容量安排和短期交易决策都依赖未来数小时的负荷估计。负荷受作息、星期、季节和气象条件共同影响，规律性强但也在漂移，短期预测的难点不在于有没有规律可循，而在于用什么特征、什么模型能把这些规律稳定地转化为预测能力。

本项目使用课程提供的城市电力负荷监测数据，完成从数据清洗、探索性分析到预测建模的全流程，重点回答三个研究问题：其一，三个计量区域的负荷在日、周、季节尺度上呈现什么结构，彼此有何差异；其二，温度、湿度等气象因素与负荷的统计关联有多强，能否转化为预测增益；其三，在一个日周期极强、且存在水平漂移的序列上，简单的历史同期基线与线性回归、梯度提升树两种学习模型相比孰优孰劣，误差在区域间和时段间如何分布。

所有实验都按同一套规则做：按时间顺序划分数据以防泄漏，特征只使用预测起点可获得的信息，模型比较与特征消融共用同一样本集，报告中的每个数字都能追溯到实验日志中的实际运行记录。数据来源、字段与清洗结果见第 2、3 章。

2 数据介绍

本项目使用课程提供的城市电力负荷监测数据，文件为单个 CSV，共 52,416 条记录、9 个字段，覆盖 2017 年 1 月 1 日 0 时至 12 月 30 日 23 时 50 分，采样间隔 10 分钟，已记录时段内无中断（数据止于 12 月 30 日，共 364 天）。

数据由两类信息组成。一类是三个区域的用电负荷，分别记为区域 1、区域 2 和区域 3；另一类是同一时刻的气象观测，包括气温、相对湿度、风速和两项辐射指标（总漫射辐射、漫射辐射）。所有记录共用同一个时间戳，三个区域的负荷与气象变量在时间上严格对齐，这使得三者可以直接比较，也便于构造滞后特征。

原始数据没有注明负荷与气象变度和百分比；风速、两项辐射量与负荷的单位则无从确认。因此本报告在涉及绝对数值时一律按原始单位陈述，不做单位换算，涉及区域间比较的结论均使用比值或百分比，不受单位影响。字段的完整说明见表 1。原始列名中，区域 2 与区域 3 的字段名含有连续两个空格，这一细节在第 3 章的数据清洗中单独处理。

表 1　数据字段说明

序号	原始列名	清洗后列名	类型	单位	最小值	最大值	均值	含义
1	DateTime	timestamp	datetime64	—	2017-01-01 00:00	2017-12-30 23:50	—	时间戳，10 分钟采样
2	Temperature	temperature	float64	℃	3.25	40.01	18.81	气温
3	Humidity	humidity	float64	%	11.34	94.80	68.26	相对湿度
4	Wind Speed	wind_speed	float64	未注明	0.0500	6.48	1.96	风速
5	general diffuse	general_diffuse_flows	float64	未注明	0.0040	1,163.00	182.70	总漫射辐射

序号	原始列名	清洗后列名	类型	单位	最小值	最大值	均值	含义
6	diffuse flows	diffuse_flows	float64	未注明	0.0110	936.00	75.03	漫射辐射
7	Zone1 Power Consumption	zone1_power	float64	未注明	13,895.70	52,204.40	32,344.97	区域 1 负荷
8	Zone2Power Consumption	zone2_power	float64	未注明	8,560.08	37,408.86	21,042.51	区域 2 负荷（原列名含连续两个空格）
9	Zone3Power Consumption	zone3_power	float64	未注明	5,935.17	47,598.33	17,835.41	区域 3 负荷（原列名含连续两个空格）

3　数据清洗与预处理

3.1　列名与类型规整

原始列名混用了空格、大小写和缩写（如 Wind Speed、general diffuse flows），其中区域 2、区域 3 的列名还带有连续两个空格。清洗时先将连续空白折叠为单个空格、去除首尾空格，再统一映射为小写下划线形式，得到 timestamp、temperature 、 humidity 、 wind_speed 、 general_diffuse_flows 、 diffuse_flows 、 zone1_power、zone2_power、zone3_power 共 9 个列名。

原始时间列形如 1/1/2017 0:00，按字符串读入时是 object 类型。用 pandas.to_datetime 解析后转为 datetime64 类型，52,416 条记录全部解析成功，没有出现无法识别的格式。其余 8 个数值字段统一转为 float64，转换过程中没有产生新的缺失值。

3.2　缺失值与重复值

原始数据不含缺失值，52,416 行×9 列均完整，因此没有做插值。完全重复的行为 0 条，重复时间戳为 0 个，时间序列严格按 10 分钟等间隔排列，也不存在跨天、跨月的采样中断。如果存在缺测时段，构造 t+6 的预测目标时会出现目标时刻无观测的情况。本数据没有这个问题，因此可以直接按固定步长构造样本。

3.3　异常值

异常值用 IQR 规则标记，即超出 Q1−3×IQR 或 Q3+3×IQR 的点。九个字段中共标记 1,708 条，其中 diffuse_flows 1,690 条、zone3_power 18 条，其余字段在 3 倍 IQR 下没有被标记。

对这些点逐类核对后，决定**只标记、不删除、不替换**。diffuse_flows 的极端点集中在 9 时至 16 时以及 2 月和 5 月，正是日照最强的时段和月份，反映的是气象变量的真实波动，删除会削掉白天的信息；zone3_power 的 18 个点集中在 7 月的 19 时至 20 时，对应夏季晚高峰，属于业务上需要重点关注的时段，尤其不能删。清洗阶段另有一项判断：负荷出现负值应视为错误，检查结果为 0 条。标记结果单独写入 anomaly_flags.csv，仅用于说明，不作为模型输入。

3.4 输出接口

清洗后的数据固定输出 9 列，行按时间升序排列、索引重置，写入 data/processed/power_load_clean.csv，列名与类型在 docs/data_interface.md 中固定下来。后续的特征工程、模型训练和误差分析统一从这个文件读取，避免各人按不同假设重复读取原始数据。清洗结果的质量指标汇总在表 2。

表 2　数据质量与清洗情况

检查项	结果	处理方式
原始记录数	52,416	—
字段数	9	—
时间范围	2017-01-01 00:00 至 2017-12-30 23:50	—
时间跨度	364 天	—
采样间隔	10 分钟（1 种）	无需重采样，保持 10 分钟粒度
缺失值	0 个	无缺失，未做插值
完全重复行	0 行	重复行整行删除
重复时间戳	0 个	无重复，不需要去重
负荷负值	0 个	无负值，未做截断
列名规整	2 处	Zone 2 / Zone 3 列名的连续空格折叠为单空格，再统一为 snake_case
类型转换失败	0 个	全部字段可转为数值，无需人工修正
异常值标记（3× IQR）	1,708 条	只标记到 anomaly_flags.csv，不删除、不替换
输出记录数	52,416	行按时间升序，索引重置
其中 diffuse_flows	1,690 条	极端点集中在固定时段，按真实波动保留
其中 zone3_power	18 条	极端点集中在固定时段，按真实波动保留

4　负荷的时序规律

4.1　年内走势

三个区域的负荷水平差距明显。年内日均值分别为 32,345、21,043 和 17,835，区域 1 约为区域 3 的 1.8 倍，且这一量级关系在年内稳定（图 1）。季节形态则三个区各不相同：区域 1 最平缓，最高月比最低月高约两成半；区域 2 在 7、8 月形成夏季峰，11、12 月又出现一个高于春季的次高峰；区域 3 起伏最剧烈，7 月达到 28,200，12 月降至 11,000，最高月是最低月的 2.6 倍。按季节平均，区域 3 夏季负荷是冬季的 1.59 倍，区域 1 和区域 2 分别只有 1.17 和 1.12 倍。

区域 3 从 9 月到 12 月的连续下滑需要谨慎对待：其中既有季节性成分，也可能包含全年负荷水平的整体下移，本数据只有一个年度周期，两种成分无法分离，因此按观测值陈述，不做季节性归因。负荷水平的年内下移是可以确认的：区域 1 在训练期的日均值为 33,065，测试期为 29,056。这一事实直接关系到第 6 章的数据划分方式。

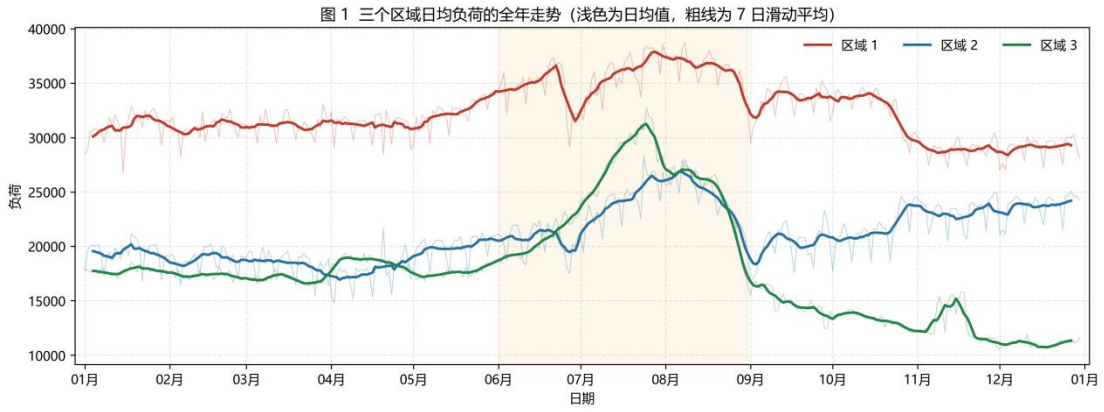


图 1　三个区域日均负荷的全年走势

2017 年 1 月 1 日至 12 月 30 日，52,416 条 10 分钟观测按日取均值；浅色线为日均值，粗线为 7 日滑动平均；阴影区间为 6—8 月。

4.2　日内规律

三个区域的峰谷时刻高度一致，谷值出现在 6 至 7 时，峰值出现在 20 时。区域 1 和区域 2 的峰谷比为 1.89，区域 3 为 2.23（图 2）。曲线呈"凌晨低谷、午间平台、晚间高峰"的三段结构，并非典型双峰。

工作日与周末的差距主要落在白天：区域 1 晚高峰时段工作日与周末仅差 2.8%，全天差距 3.6%；区域 2 差别更明显（6.2% 与 8.2%）；区域 3 几乎没有区别。峰值时刻的日间稳定性很高：三个区分别有 98.9%、97.5%、99.2% 的日子全天最高负荷落在 18 至 21 时。日峰值本身的波动则差别明显：变异系数区域 1 为 7.4%、区域 2 为 12.5%、区域 3 高达 27.7%——这一差别解释了第 6 章分区域误差的分布：区域 3 难预测，主因不在模型，而在目标序列自身的波动幅度。

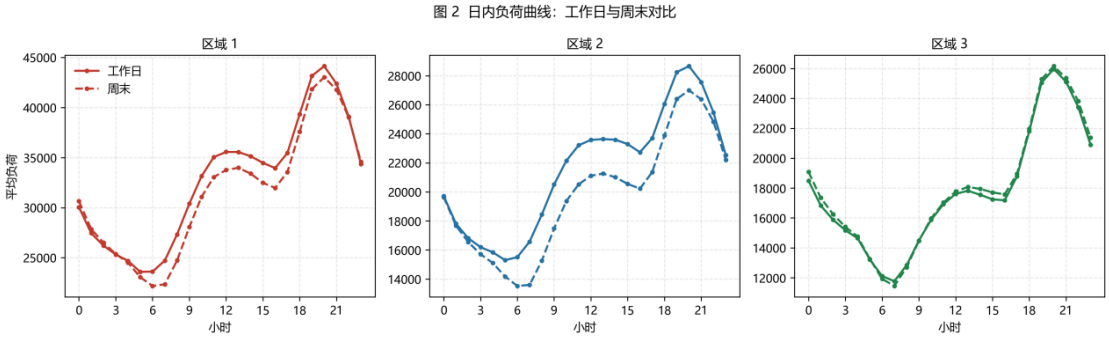


图 2　日内负荷曲线：工作日与周末对比

已记录时段内观测按小时取均值，实线为工作日，虚线为周末；三个子图的纵轴范围各自独立，不宜直接比较幅度。

4.3　周内规律

区域 1 和区域 2 的周内走势一致：周一至周四持平，周五开始走弱，周日最低，工作日与周末的比值分别为 1.04 和 1.09。区域 3 的走向相反，周五反而是全周最低，工作日与周末比值 0.99，实际不存在星期效应（图 3）。这一差异决定了星期特征对区域 3 的贡献有限。

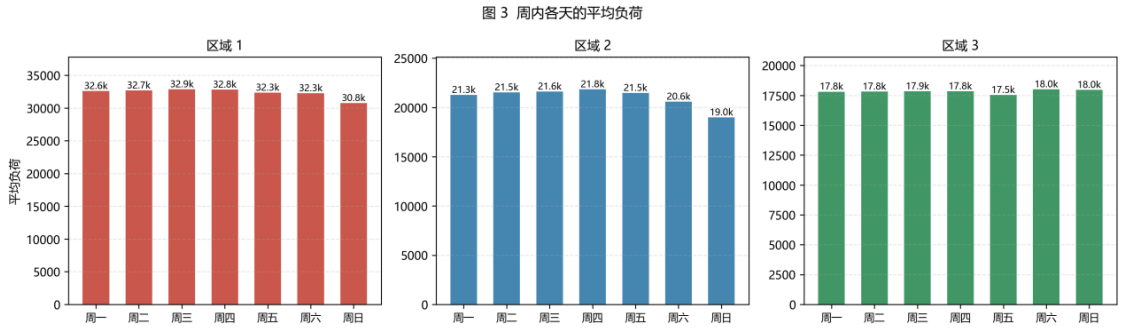


图 3　周内各天的平均负荷

已记录时段内观测按星期取均值；三个子图纵轴范围独立，跨区域比较请参见第 4.5 节汇总表。

4.4　季节规律

按月统计的箱线图显示，中位数的月度走势与日均值一致，但箱体高度在夏季明显变大，说明夏季不仅负荷水平高，日内波动和日间差异也都大于冬季（图 4）。各区极值月份：区域 1、2 最高月在 8 月，区域 3 在 7 月；最低月区域 1、3 分别为 11、12 月，区域 2 的低谷却出现在 4 月——该区域冬季负荷并不低，真正的低谷在春季。

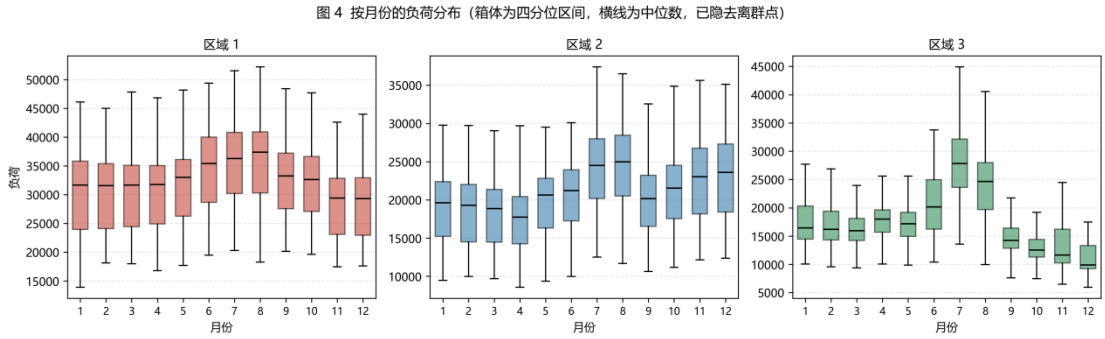


图 4　按月份的负荷分布

已记录时段内 10 分钟观测按月份绘制的箱线图，箱体为四分位区间，横线为中位数，须线延伸至 1.5 倍四分位距，离群点已隐去；三个子图纵轴范围独立。

4.5　分区对比

指标	区域 1	区域 2	区域 3

年内日均	32,345	21,043	17,835
峰时 / 谷时	20 时 / 6 时	20 时 / 6 时	20 时 / 7 时
峰谷比	1.89	1.89	2.23
周末相对工作日	−3.6%	−8.2%	+1.2%
夏冬比	1.17	1.12	1.59

三个区共享同一套时间结构，差别集中在幅度上：区域 1 量级最大、曲线最平稳；区域 2 星期效应最强且冬季有次高峰；区域 3 量级最小，季节性和日内波动都最强。建模时应当区别对待，而不是对三个区套用同一组特征。

5　天气因素与负荷的关系

5.1　相关系数

三个区域负荷与五个气象因子的相关系数见图 5 和下表，括号外是 Pearson 系数，括号内是 Spearman 系数。

因子	区域 1	区域 2	区域 3
温度	+0.44（+0.43）	+0.38（+0.38）	+0.49（+0.44）
湿度	−0.29（−0.30）	−0.29（−0.31）	−0.23（−0.21）
风速	+0.17（+0.11）	+0.15（+0.09）	+0.28（+0.08）
总漫射辐射	+0.19（+0.26）	+0.16（+0.24）	+0.06（+0.08）
漫射辐射	+0.08（+0.12）	+0.04（+0.10）	−0.04（−0.02）

温度是唯一在三个区域都稳定为正的因子。风速和总漫射辐射的两种系数差得不小，例如区域 3 风速的 Pearson 系数是 0.28，Spearman 系数只有 0.08，说明相关性主要靠少数极端时段撑起来，直接放进线性模型并不稳妥。三个区域负荷之间的相关系数在 0.75 到 0.84 之间，说明它们受共同因素驱动。

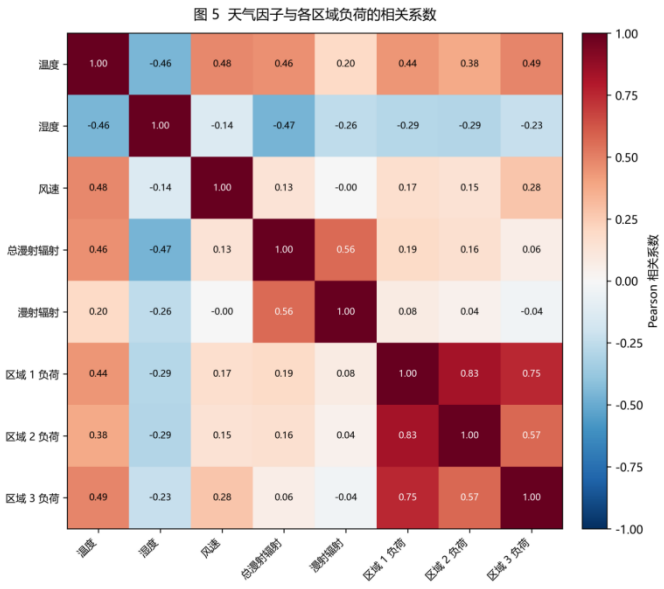


图 5　天气因子与各区域负荷的相关系数

已记录时段内 52,416 条观测计算的 Pearson 相关系数；色标固定为 −1 至 1，对角线为变量自身相关，数值恒为 1。

5.2 温度与负荷的分段特征

把温度分箱后看均值（图 6）：6 ℃ 到 33 ℃ 之间负荷随温度上升，33 ℃ 以上进入平台。温度从 6.5 ℃ 升到 33.5 ℃ 时，区域 1、2 的分箱均值各上升 69%，区域 3 上升 158%，与前面夏冬比 1.59、峰谷比 2.23 的结果对得上。低温一侧则**没有观察到典型的 U 形**：3 ℃ 到 5 ℃ 的负荷并不高于 20 ℃ 附近，说明这三个区域的用电对低温不敏感，温度特征在冬季帮不上多少忙。

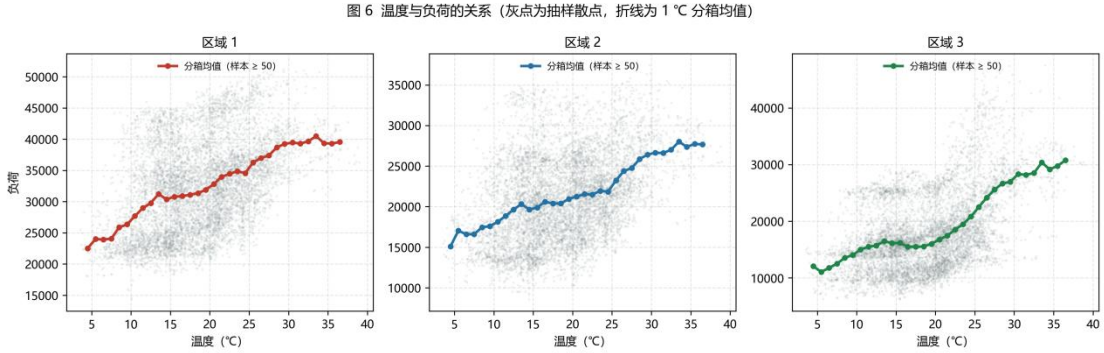


图 6 温度与负荷的关系

灰点为从已记录时段观测中随机抽样的 8,000 条记录，折线为按 1 ℃ 分箱后的负荷均值，仅保留样本量不少于 50 的区间；三个子图纵轴范围独立。

5.3 与消融结果的呼应

这里可以先和第 6 章的消融实验对一下：温度相关性最高的区域 3（Pearson 系数 0.49），天气增益反而最小；而区域 1 的增益最大。也就是说，相关只说明两者存在统计关联，能不能变成预测增益，还要看模型形式，以及特征之间有多少信息重叠。具体数值放在 6.5 节一起说。

5.4 局限

这些只是统计共变，不能当作因果。节假日没有单独建模，昼夜没有区分，相关系数也没做去趋势，年内负荷整体下移会让系数略微偏低。

6　短期负荷预测：模型、消融与误差分析

6.1　预测任务与实验协议

预测任务是**提前 1 小时**：对预测起点 t，要预测的是其后第 6 个时间步（t+6）的负荷。三个区域各自建样本、各自训练和评价。数据按预测起点的时间顺序分成训练、验证、测试三部分，比例 70%/15%/15%，三个区域共用同一套切分。实际边界是：训练集到 2017-09-14 19:40，共 35,975 个起点；验证集到 11-07 08:40，共 7,704 个；测试集到 12-30 22:50，共 7,711 个。

目标时刻在起点之后一小时，只按起点切分会有问题：子集末尾几个起点的目标时刻会落到下一个子集的起点区间里，训练集末尾的标签等于被验证集提前看到，验证集末尾也一样。为此，训练集和验证集末尾各剔除 6 个起点，共 12 个样本不进入任何子集。剔除之后，训练末起点的目标时刻是 09-14 20:40，早于验证集首起点的 09-14 20:50；验证末起点的目标时刻是 11-07 09:40，早于测试集首起点的 11-07 09:50，相邻子集的标签和起点区间不再重叠。这一检查已写进 verify_against_plan() 和自动化测试。测试集区间不受影响，历史同期基线的指标与剔除前完全一致。

特征只用预测起点 t 以及更早的观测，滚动统计一律向过去取窗口，标准化这类需要拟合的预处理只在训练集上拟合。验证集用来选模型和参数，测试集只评价一次。天气特征取的是起点时刻的同步观测，这是实验约定，实际部署时应换成天气预报。

6.2　特征与模型

用的特征可以分成三类。第一类是历史负荷，包括当前负荷、1 小时前、24 小时前和 7 天前的负荷（对应字段 load_lag_0/6/144/1008），其中最关键的是与预测目标时刻对齐的周同期负荷 load_lag_target_week = y(t−1002)。这个特征的用意很直接：负荷的日周期和周周期都很稳，97%以上的日子峰值落在 18—21 时（见 4.2 节），把目标时刻上一周同期的负荷告诉模型，等于把最强的周期信息交给它；而 t−1002 远早于预测起点，取值在起点时已经知道，不涉及未来信息。第二类是时间特征，包括小时、星期、月份及其周期编码。第三类是天气特征，取起点时

刻的温度、湿度、风速和两项辐射。历史同期基线不使用这些特征，直接取目标时刻的上一周同期值 y(t+6−1008)，也就是 y(t−1002)。

模型一共三个。**历史同期基线**不需要训练，也没有参数；**线性回归**先做标准化再最小二乘拟合，同样没有需要调的参数；**梯度提升树**用 scikit-learn 的 HistGradientBoostingRegressor，在验证集上对一小组参数网格选优，随机种子固定为 42。评价指标是 MAE、RMSE 和 MSE，按模型和区域分别在测试集上计算，另外给出三个区域合并的结果。

6.3 模型比较

表 3　三个模型在测试集上的对比（最终特征配置）

注：测试集为 2017-11-07 09:50 至 2017-12-30 22:50，每区域 7,711 个样本；加粗为同区域最优；MSE 按逐点预测误差平方的均值计算，RMSE 为其平方根，各指标均由未舍入结果计算后再显示舍入。

模型	Z1 MAE/RMSE	Z2 MAE/RMSE	Z3 MAE/RMSE	合并 MAE/RMSE	合并 MSE
历史同期基线	831.04/ 1151.45	966.29/ 1366.37	1354.70/ 2216.34	1050.68/ 1643.67	2,701,655
线性回归	**547.15/ 777.86**	**513.58/ 729.94**	**481.52/ 792.29**	**514.09/ 767.16**	**588,534**

梯度提升树	925.93/ 1201.90	762.60/ 1118.37	1103.33/ 1434.54	930.62/ 1258.73	1,584,408

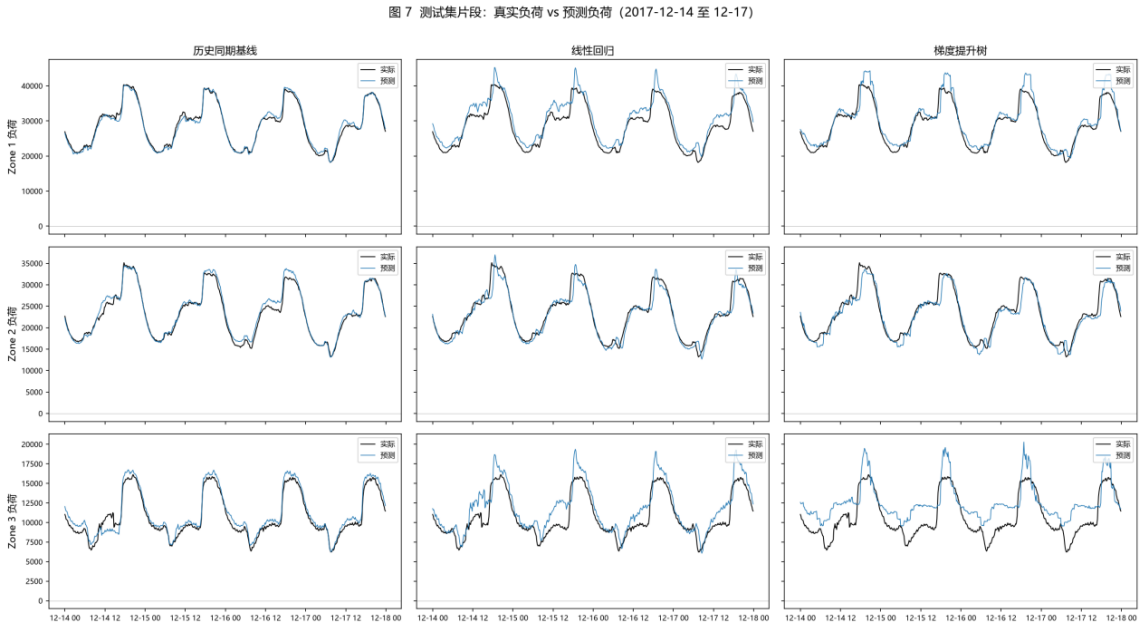


图 7　测试集片段：真实负荷与预测负荷对比

2017 年 12 月 14 日至 12 月 17 日的局部片段，三行分别为三个模型、三列为三个区域；完整测试区间的指标见表 3。

三个指标的侧重不同：MAE 看平均绝对偏差，RMSE 和 MSE 对大误差更敏感，因为平方会放大极端值，三者排序一致时结论更可靠。全部指标数值见 outputs/metrics/error_overall_tw.csv。

线性回归在三个区域的 MAE 和 RMSE 上都是三者中最低，合并后为 514.09/767.16，比基线下降 51.1%/53.3%。梯度提升树次之，合并为 930.62/1258.73，比基线下降 11.4%/23.4%，同样全面优于基线。线性回归的优势来自两点：一是它和基线一样握有目标时刻的上一周同期值，周期信息不缺；二是它还能用温度、小时等特征在这个基础上做修正。测试期负荷水平比训练期整

体下移，区域 1 日均从 33,065 降到 29,056，区域 3 从 20,052 降到 11,949，基线锚定上周值，应对不了这种漂移，学习模型则能吸收一部分。

6.4 特征贡献：目标时刻对齐的周同期特征

为单独看这一特征的贡献，样本、切分、模型配置和评价指标都不动，只切换是否包含 load_lag_target_week = y(t−1002)，结果见表 4。这个特征用的信息和历史同期基线同源，但学习模型还能叠加天气、小时等特征做修正，所以加入之后理应不弱于基线。

表 4　加入目标对齐周同期特征前后的测试集指标对比

注：样本、切分、模型配置与评价指标与主实验完全一致，唯一差别是否包含 load_lag_target_week = y(t−1002)；MSE 为三区合并口径。

模型	Z1 MAE/RMSE	Z2 MAE/RMSE	Z3 MAE/RMSE	合并 MAE/RMSE	合并 MSE
线性回归（不含周同期）	2143.03/ 2512.61	1016.45/ 1550.16	1321.49/ 1666.97	1493.66/ 1957.47	3,831,677
线性回归（含周同期）	**547.15/ 777.86**	**513.58/ 729.94**	**481.52/ 792.29**	**514.09/ 767.16**	**588,534**

梯度提升树（不含周同期）	1596.96/2255.25	1091.57/1728.73	1840.15/2233.54	1509.56/2086.73	4,354,460
梯度提升树（含周同期）	**925.93/1201.90**	**762.60/1118.37**	**1103.33/1434.54**	**930.62/1258.73**	**1,584,408**

加粗为含周同期特征的一组。

表 4 显示这一项特征的贡献很大：加入后，线性回归三区测试集 RMSE 比未加入时下降 52.5%–69.0%，比历史同期基线下降 32.4%–64.3%，合并口径 MAE/RMSE 由 1493.66/1957.47 降到 514.09/767.16，降幅 51.1%/53.3%；梯度提升树三区 RMSE 比基线低 18.2%–35.3%，合并口径低 11.4%/23.4%。也就是说，两个学习模型拿到周期信息之后都超过了基线，同时还保留了用其他特征修正偏差的能力。

6.5　天气特征消融

消融实验在推荐特征集上做，也就是已经包含周同期特征的那一组。A 组只有历史负荷和时间特征，B 组在 A 组基础上加天气特征，其余样本、切分、模型和参数都一样。

表 5　天气特征消融结果（推荐特征配置）

注：数值为含天气特征组（B 组）相对不含天气特征组（A 组）的测试集 RMSE 变化，负值表示误差下降。

模型	区域	测试集 RMSE 变化
线性回归	Z1/Z2/Z3	−10.99%/+5.08%/ +0.98%
梯度提升树	Z1/Z2/Z3	−17.85%/−5.94%/ −0.43%

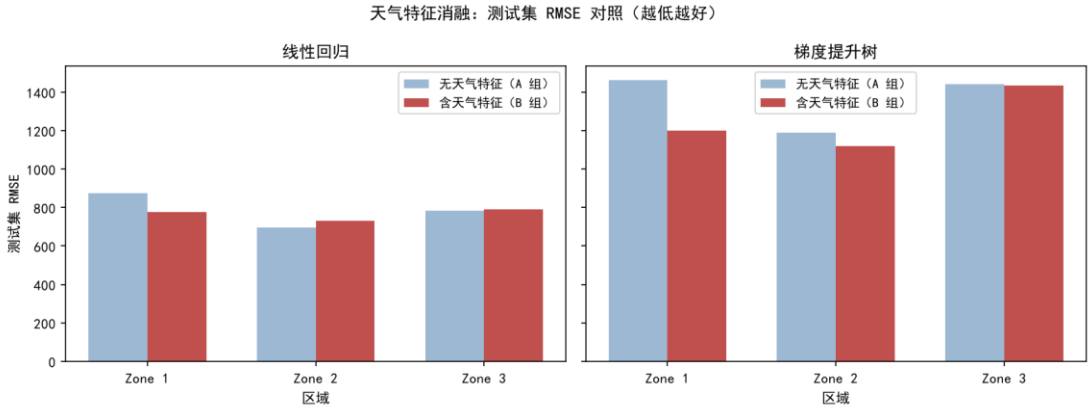


图 9　天气特征消融：推荐配置下的测试集 RMSE 对照

六个（模型 × 区域）组合的测试集 RMSE，柱高越低越好；深色为含天气特征组（B 组），浅色为不含天气特征组（A 组）；幅度比较见表 5。

天气特征能带来多少增益，要看模型和区域。梯度提升树三区 RMSE 全部下降，幅度 0.4%到 17.9%，区域 1 降幅最大；线性回归只在区域 1 下降 11.0%，区域 2、3 基本持平，分别上升 5.1%和 1.0%。所以对梯度提升树来说，天气特征是稳定增益；对线性回归，它接近中性。另外，这里用的天气特征是预测起点的同步观测，实际部署时要换成天气预报，增益很可能还会再打折扣。

6.6　分区域与高峰时段误差分析

下面的分析都基于推荐模型，也就是含周同期特征的那一组。三个区域量级不同，直接比 RMSE 不公平，所以先看相对误差 nRMSE：线性回归为 2.68%/3.10%/6.63%，梯度提升树为 4.14%/4.75%/12.01%，历史同期基线为 3.96%/5.81%/18.55%。无论用哪个模型，区域 3 都是最难的，与第 4 章的发现一

致，它的日峰值变异系数达到 27.7%。线性回归在三个区域都是最优，把区域 3 的相对误差从基线的 18.55%压到了 6.63%。

再看系统性偏差，也就是预测值减实际值的均值。线性回归在三个区域分别为+86/−223/−173，梯度提升树为+522/−271/+549，基线为−17/−109/+108，方向并不一致。线性回归的偏差量级最小，绝对值也远低于它的 MAE，不存在明显的单向高估或低估。

高峰时段的划分只依赖训练集：取训练集样本目标负荷 y(t+6)的 90%分位数作为阈值，计算区间是训练集的目标时间范围，即 2017-01-08 01:00 到 2017-09-14 20:40，不含只用作历史特征来源的前 7 天，也不含验证集和测试集的任何时刻。测试样本中，目标负荷达到或超过阈值的算高峰时段，其余算普通时段。按这个口径，区域 2 高峰时段梯度提升树的 RMSE 为 1736，是普通时段 826 的 2.1 倍，偏差也由−100 变成−799，说明模型在高峰时段系统性低估峰值，这是对调度最不利的误差方向，在推荐配置下依然如此。区域 1 测试期高峰样本只有 9 条，区域 3 为 0 条，原因是测试期负荷整体下移，这两个区域的高峰表现无法从本次数据确认。

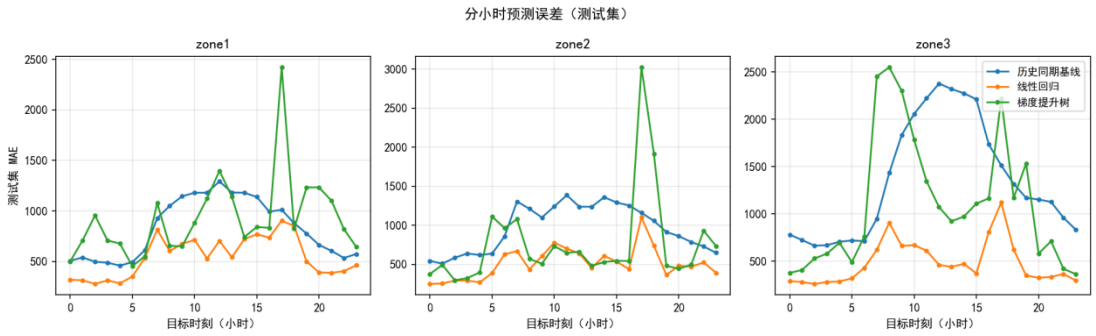


图 8（a）　分小时预测误差（测试集）

按目标时刻的小时统计 MAE，三个区域各一子图；基于含周同期特征的推荐模型。

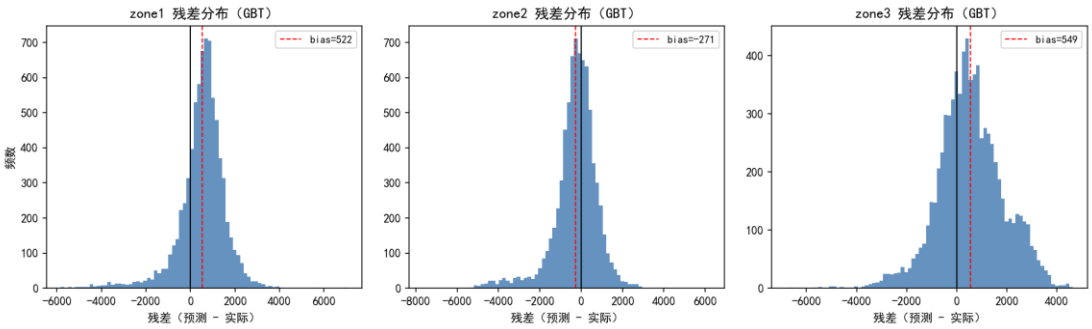


图 8（b） 残差分布（梯度提升树）

残差 = 预测 − 实际，横轴为残差，纵轴为频数；三个区域各一子图。

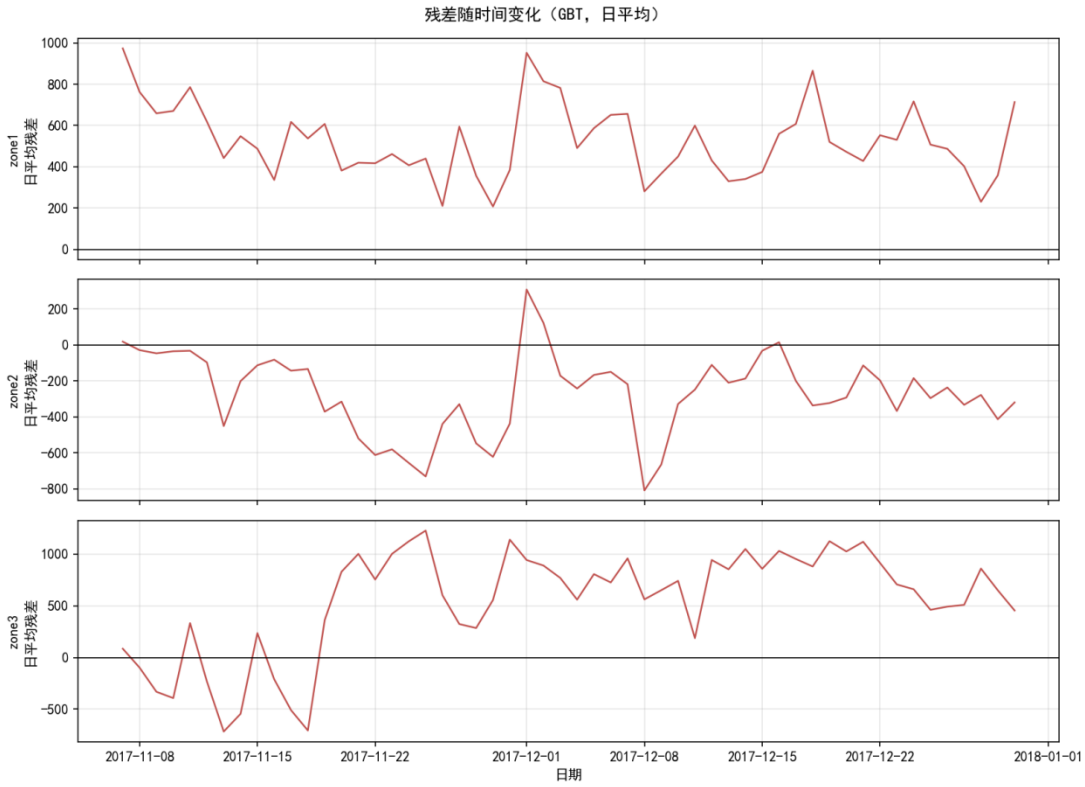


图 8（c） 残差随时间变化（梯度提升树，日平均）

测试集残差按日取均值后按日期排列，用于检查误差是否随时间漂移。

7　总结与结论

7.1　主要发现

1. **日周期和周周期是短期预测的主导规律，周同期特征带来的改进最大。** 线性回归合并 MAE/RMSE 为 514.09/767.16，比基线下降 51.1%/53.3%，比梯度提升树低 44.8%/39.0%，三个区域都领先。原因并不复杂：学习模型拿到了与基线同源的周期信息，又能用其他特征修正偏差，不需要复杂模型就能明显好过基线。

2. **预测难度的区域差异主要来自数据本身。** 区域 3 的 nRMSE 仍是三区最高，线性回归 6.63%、基线 18.55%，与它 27.7%的日峰值变异系数一致。所以跨区域比较要用相对指标，不能只看 RMSE。

3. **天气特征的增益因模型和区域而异。** 梯度提升树三区 RMSE 全部下降，线性回归只在区域 1 改善明显，区域 2 和 3 基本持平。增益大小与温度相关性的排序也对不上：相关性最高的区域 3，增益反而最小。所以相关不等于增益，不能笼统地说“天气特征能改善预测”。

4. **高峰时段更难预测，而且误差方向不利。** 区域 2 的高峰误差是普通时段的 2.1 倍，模型系统性低估峰值，偏差由−100 转为−799；区域 1 和 3 的高峰测试样本不足，无法确认。

7.2　对电网调度的建议

如果用于小时级调度，优先用**带周同期特征的线性回归**，历史同期基线留作不训练、无维护成本的兜底。高峰时段按训练集九十分位阈值划分，实测集中在 17–22 时，模型在这里倾向低估，裕度应往高估方向留，区域 2 尤其如此；区域 3 波动最大，建议单独监控并多留备用容量。天气特征接入天气预报后的收益有限，先把周同期特征和实时负荷接入做扎实，再考虑天气投入。

7.3　局限

数据只覆盖一个城市、一年（364 天，缺 12 月 31 日），结论不能外推为跨城市、跨年度的一般规律。具体说，区域 3 在 9–12 月的下滑含趋势成分，无法

与季节性分开；没有观察到低温 U 形回升，采暖季是否同样有效无法判断；节假日和极端天气样本极少，这些情形没有验证。所有对比只报数值和相对变化，没有做显著性检验，因此不使用“显著”一类措辞。

方法上还有一点限定：周同期特征的大幅改善依赖日周周期的稳定性，本数据 97%以上的日子峰值落在 18–21 时，换成作息规律较弱、负荷波动更大的场景，增益预计会小一些；遇到检修、节假日这类日程突变，基线和线性回归也都会失效，需要另外加入日程特征。各条结论对应的证据和图表见 docs/results_conclusions.md 的对照表。

8　小组成员分工与个人承担

8.1　分工表

成员	学号	承担章节/模块	具体工作内容
郑宜桥	25121628	第 2、3、4、5 章；数据模块	数据审计、清洗与接口定义、日/周/季节规律分析、天气与负荷关系分析、表 1 与表 2
况子昱	25121631	第 6 章；模型模块	预测协议、时间切分与特征、基线与梯度提升模型、模型比较与交付检查
徐启翔	25121632	摘要、第 1、7 章；消融与误差分析	报告攥写、天气特征消融、分区域与高峰时段误差分析、PPT 制作与汇报

8.2　个人承担说明

此处暂略，个人填写。

9　附录

附录 A　代码仓库

GitHub 仓库：https://github.com/77xiang/python_data_processing

代码结构：src/data_cleaning.py（清洗）→ src/features.py、src/splits.py（特征与切分）→ src/models.py、src/evaluate.py、src/compare.py（模型与比较）→ src/ablation.py（消融）→ src/error_analysis.py（误差分析）→ src/probe_target_week.py（特征贡献分析）；分析图表分别由 src/eda.py、src/weather_analysis.py 生成。

附录 B　主要代码片段

以下为与正文第 3 章、第 6 章对应的主要代码片段。为控制篇幅，省略了部分文档字符串、注释与校验代码，并对个别长行按 Python 语法折行；完整实现见附录 A 所列仓库。

B.1 数据清洗：列名规整与异常值标记（src/data_cleaning.py）

```
def _normalize_name(name) -> str:
    # 折叠连续空白、去首尾空格、转小写，用于匹配原始列名
    return re.sub(r"\s+", " ", str(name)).strip().lower()



def standardize_columns(df: pd.DataFrame) -> pd.DataFrame:
    mapping = {}
    for col in df.columns:
        key = _normalize_name(col)
        mapping[col] = RAW_COLUMN_MAP.get(key, _snake_name(col))
    out = df.rename(columns=mapping)
    if out.columns.duplicated().any():
        dup = out.columns[out.columns.duplicated()].unique().tolist()
        raise ValueError(f"列名规整后出现重复：{dup}")
    return out



def iqr_flags(
    df: pd.DataFrame,
    columns: list[str] | None = None,
    k: float = 3.0,
```

```
) -> pd.DataFrame:
    # 按 k*IQR 规则标记极端点，只标记，不删除、不修正任何行
    cols = columns or [
        c for c in LOAD_COLUMNS + WEATHER_COLUMNS if c in df.columns
    ]
    records = []
    for col in cols:
        s = pd.to_numeric(df[col], errors="coerce")
        q1, q3 = s.quantile(0.25), s.quantile(0.75)
        iqr = q3 - q1
        if pd.isna(iqr) or iqr <= 0:
            continue
        lower, upper = q1 - k * iqr, q3 + k * iqr
        mask = (s < lower) | (s > upper)
        for idx in s.index[mask.fillna(False)]:
            records.append({
                "timestamp": df.loc[idx, "timestamp"],
                "column": col,
                "value": float(s.loc[idx]),
                "lower_bound": float(lower),
                "upper_bound": float(upper),
            })
    return pd.DataFrame.from_records(records)
```

B.2 时间切分与边界剔除（src/splits.py）

```
FORECAST_HORIZON = 6      # 提前 1 小时 = 6 个时间步（10 分钟采样）

WEEK_STEPS = 1008

PURGE = FORECAST_HORIZON  # 子集末尾剔除的起点数




def origin_bounds(n_rows: int) -> tuple[int, int]:

    # 下界保证 y(t-1008) 存在，上界保证目标 y(t+6) 存在

    return WEEK_STEPS, n_rows - 1 - FORECAST_HORIZON




def split_ranges(n_rows: int) -> dict[str, tuple[int, int]]:

    first, last, raw_train_end, raw_val_end = _raw_boundaries(n_rows)

    return {

        "train": (first, raw_train_end - PURGE),

        "val": (raw_train_end + 1, raw_val_end - PURGE),

        "test": (raw_val_end + 1, last),

    }




def purge_ranges(n_rows: int) -> dict[str, tuple[int, int]]:

    _, _, raw_train_end, raw_val_end = _raw_boundaries(n_rows)

    return {

        "train_tail": (raw_train_end - PURGE + 1, raw_train_end),

        "val_tail": (raw_val_end - PURGE + 1, raw_val_end),

    }
```

```
def split_origin_indices(n_rows: int) -> dict[str, np.ndarray]:
    ranges = split_ranges(n_rows)
    return {
        name: np.arange(start, end + 1, dtype=int)
        for name, (start, end) in ranges.items()
    }
```

B.3 样本与周同期特征构造（src/features.py）

```
HISTORY_LAGS = {
    "load_lag_0": 0,        # 当前负荷 y(t)
    "load_lag_6": 6,        # 1 小时前
    "load_lag_144": 144,    # 24 小时前同期
    "load_lag_1008": 1008,  # 7 天前同期
}
TARGET_WEEK_LAG = WEEK_STEPS - FORECAST_HORIZON   # 1002 = 1008 - 6

# build_zone_samples() 内：origins 为预测起点行号，load 为该区域负荷序列
for name, k in HISTORY_LAGS.items():
    X[name] = load[origins - k]          # 全部 <= t，不含未来

# 与预测目标时刻对齐的周同期负荷：y(t+6-1008) = y(t-1002)
if include_target_week:
```

```
    X[TARGET_WEEK_LAG_NAME] = load[origins - TARGET_WEEK_LAG]


y = pd.Series(
    load[origins + FORECAST_HORIZON], index=origins, name="target"
)
meta["origin_timestamp"] = ts_arr[origins]
meta["target_timestamp"] = ts_arr[origins + FORECAST_HORIZON]
meta["split"] = split_labels(origins, n)
```

B.4 高峰阈值与高峰/普通时段划分（src/error_analysis.py）

```
def train_thresholds() -> dict[str, float]:
    # 训练起点 + 预测时域 = 训练集目标行，不含前 1008 行与验证/测试集
    df, _ = clean_data()
    origins = split_origin_indices(len(df))
    idx = origins["train"] + FORECAST_HORIZON
    out = {}
    for z in ZONES:
        out[z] = float(df.loc[idx, f"{z}_power"].quantile(PEAK_QUANTILE))
    return out



# peak_vs_normal() 内：按测试样本的目标负荷与该区域阈值比较
for (model, zone), g in pred.groupby(["model", "zone"]):
    peak = g["actual"] >= thresholds[zone]
    for flag, sub in ((True, g[peak]), (False, g[~peak])):
```

```
            if len(sub) == 0:      # 一侧无样本则不输出该行
                continue
            rows.append({
                "model": model, "zone": zone,
                "period": "peak" if flag else "normal",
                "n_samples": len(sub),
                "rmse": np.sqrt((sub["error"] ** 2).mean()),
                "bias": sub["error"].mean(),
            })
```